

**PROCEDIMIENTO DE ACCIONES CORRECTIVAS, PREVENTIVAS Y DE MEJORAMIENTO DEL
SOFTWARE**

RENTAMOVIL

**Thiago Fabian Rojas Guevara — David Santiago Erez Rodríguez — Nicole Dayana Ramírez
Vargas**

ANÁLISIS Y DESARROLLO DE SOFTWARE 3145556

SERVICIO NACIONAL DE APRENDIZAJE

2026

Marco de referencia:

Este procedimiento adopta la terminología y el ciclo de gestión de problemas de la norma ISO/IEC 29110 (perfil básico para pequeñas organizaciones de desarrollo de software — VSE), sin constituir una certificación formal. Se usa como marco de referencia técnico, adaptado al alcance real del proyecto.

1. CONTROL DE CAMBIOS Y VERSIONES

Versión	Fecha	Autor	Descripción del cambio
1.0	2026-10-09	David Erez	Versión inicial del procedimiento: ciclo CAPA, plantillas de registro y hallazgos reales identificados durante el repaso técnico integral del proyecto.

2. OBJETIVO Y ALCANCE

2.1 Objetivo

Establecer un procedimiento formal y repetible para identificar, registrar, priorizar, corregir y prevenir hallazgos (defectos, incidentes y oportunidades de mejora) del software RentaMóvil a lo largo de su ciclo de vida, garantizando que cada acción quede documentada, tenga un responsable y una verificación de eficacia.

2.2 Alcance

Este procedimiento aplica a todo el ciclo de vida del aplicativo web y móvil de RentaMóvil, incluyendo:

- Los microservicios backend (`rtm-iam`, `rtm-fleet-maintenance`, `rtm-booking-reservation`, `rtm-payment-billing`, `rtm-api-gateway`) y sus repositorios de migración de base de datos.
- El front-end web (React + Vite) y la aplicación móvil (Expo + React Native).
- La infraestructura de ejecución (contenedores Docker, base de datos SQL Server compartida).
- No sustituye el proceso de gestión de incidentes operativos ya documentado en `rtm-docs/13-operations/incident-management.md`; lo complementa, enfocándose en el registro y seguimiento de las acciones correctivas/preventivas/de mejora derivadas de los hallazgos, no en la respuesta en caliente durante un incidente.

3. PROCEDIMIENTO DEL CICLO CAPA

3.1 Cómo se reporta un hallazgo

Un hallazgo puede originarse por cualquiera de estas vías:

- **Reporte de usuario o tester:** errores observados al usar la app web o móvil (ej. capturas de consola, mensajes de error reproducidos).
- **Alertas de monitoreo o logs:** errores capturados en los logs de los servicios backend o en el healthcheck del API Gateway.
- **Revisión técnica / repaso de código:** inconsistencias encontradas al auditar el código o la documentación (brechas documentación-vs-implementación, deuda técnica).
- **Incidente de producción:** cualquier evento clasificado y cerrado bajo el runbook de `rtm-docs/13-operations/incident-management.md` que requiera una acción de seguimiento.

Todo hallazgo se registra usando la plantilla de la sección 4, asignándole un número consecutivo (C-xx para correctivas, P-xx para preventivas, M-xx para mejoras).

3.2 Cómo se evalúa y prioriza

Cada hallazgo se clasifica según su impacto real sobre la operación del sistema:

Prioridad	Criterio	Tiempo de respuesta objetivo	Ejemplo real del proyecto
Crítica	Bloquea una operación de negocio clave para todos los usuarios (login, reservas, pagos) o compromete la seguridad.	Inmediato (mismo día)	Bucle infinito de login al reservar (rotación de refresh token no persistida)
Alta	Afecta una funcionalidad importante pero existe una vía alterna, o afecta a un subconjunto de usuarios.	1–2 días hábiles	Catálogo móvil mostrando vehículos no disponibles (409 al reservar)
Media	Afecta la experiencia de uso sin bloquear la operación (UX, datos faltantes, inconsistencias menores).	Dentro del sprint actual	QR de cuenta bancaria oculto en la UI
Baja	Mejora o deuda técnica que no afecta la operación actual.	Backlog / siguiente sprint	Completar <code>risks.md</code> , agregar CI/CD

3.3 Flujo del ciclo CAPA

Detección → Registro → Clasificación (prioridad) → Análisis de causa raíz → Definición de la acción (correctiva, preventiva o de mejora) → Ejecución → Verificación de eficacia → Cierre.

Una acción correctiva responde a un hallazgo que ya ocurrió (eliminar la causa de un problema real).

Una acción preventiva responde a un riesgo identificado antes de que se materialice (eliminar la causa

de un problema potencial). Una acción de mejora no corrige ni previene un fallo, sino que aumenta la calidad, robustez o mantenibilidad del software sin que exista un defecto detrás.

4. FORMATOS DE REGISTRO (PLANTILLAS EN BLANCO)

Estas plantillas se diligencian cada vez que el equipo detecta un nuevo hallazgo. Una fila/ficha por hallazgo.

4.1 Plantilla — Acción Correctiva

N° de registro	Fecha de detección	Descripción del hallazgo	Tipo / Prioridad

Causa raíz identificada:	
Acción correctiva propuesta:	
Responsable:	
Fecha compromiso:	
Fecha de cierre real:	
Verificación de eficacia (cómo se comprobó que funcionó):	
Estado (Abierta / En progreso / Cerrada):	

4.2 Plantilla — Acción Preventiva

N° de registro	Fecha de detección	Descripción del hallazgo	Tipo / Prioridad

Causa raíz identificada:	
Acción preventiva propuesta:	
Responsable:	
Fecha compromiso:	
Fecha de cierre real:	

Verificación de eficacia (cómo se comprobó que funcionó):	
Estado (Abierta / En progreso / Cerrada):	

4.3 Plantilla — Acción de Mejora

N° de registro	Fecha de detección	Descripción del hallazgo	Tipo / Prioridad

Causa raíz identificada:	
Acción de mejora propuesta:	
Responsable:	
Fecha compromiso:	
Fecha de cierre real:	
Verificación de eficacia (cómo se comprobó que funcionó):	
Estado (Abierta / En progreso / Cerrada):	

5. REGISTRO REAL — HALLAZGOS IDENTIFICADOS (LÍNEA BASE)

Estas son las acciones ya identificadas y, en su mayoría, ya ejecutadas durante el repaso técnico integral del proyecto (octubre de 2026). Sirven como línea base del procedimiento y como ejemplo de diligenciamiento de las plantillas de la sección 4. El detalle completo con archivo/línea de cada hallazgo está en `000-notes/repaso-evaluacion-rentamovil/21-BUGS-E-INCONGRUENCIAS-ENCONTRADAS.md`.

5.1 Acciones Correctivas

N°	Hallazgo	Causa raíz	Acción correctiva ejecutada	Estado
C-01	Login respondía 400 en la app móvil.	El cliente enviaba {email, password}; rtm-iam exige {identifier, password}.	Se corrigió el payload en authService.ts para usar identifier.	Cerrada

N°	Hallazgo	Causa raíz	Acción correctiva ejecutada	Estado
C-02	Registro respondía 400 en la app móvil.	El cliente enviaba campos en snake_case (first_name); el backend espera camelCase (firstName).	Se ajustó el mapeo de campos en authService.ts.	Cerrada
C-03	Consulta de perfil fallaba (ruta inexistente).	El cliente llamaba /auth/me, que se recortaba a /me por el stripping de prefijo /auth; la ruta real es /users/me.	Se corrigió el enrutamiento en env.ts (me y changePassword) para no pasar por el strip de /auth.	Cerrada
C-04	Bucle infinito de 'volver al login' al crear una reserva.	El refresh token rota en cada uso (de un solo uso); el cliente no persistía el nuevo token recibido, provocando reuso de un token ya consumido y revocación de todas las sesiones.	Se corrigió httpClient.ts y AuthContext.tsx para persistir el refreshToken rotado tras cada renovación.	Cerrada
C-05	Reservas fallando con 409 'vehículo no disponible' repetidamente.	El catálogo móvil nunca implementó el filtro de disponibilidad por sucursal/fechas (sí existía en el front web).	Se agregó branchId a Vehicle, se conectó getAvailability() y se filtró en runSearch (HomePage.tsx).	Cerrada
C-06	Rutas de pagos, reservas y seguros con nombres equivocados.	Desalineación entre las rutas asumidas por el cliente y las rutas reales expuestas por cada microservicio.	Se corrigieron las rutas en paymentService.ts, reservationService.ts e insuranceService.ts.	Cerrada
C-07	Mappers del cliente asumían campos inexistentes en el dominio real (accountNumber, tag, etc.).	Los DTO del cliente se construyeron antes de verificar el contrato real del backend.	Se reescribieron banckAccountMapper.ts e insuranceMapper.ts contra el dominio real.	Cerrada
C-08	El código QR de la cuenta bancaria no se mostraba en la app móvil.	El renderizado del <Image> del QR estaba comentado en BankAccountSelector.tsx.	Se reactivó el renderizado del QR, con aviso de que las imágenes actuales son de prueba.	Cerrada
C-09	liquibase changelogSync marcó 4 changesets como ejecutados sin ejecutarlos (rtm-booking-db).	Se usó changelogSync para saltar un único changeset bloqueado por una FK externa, pero marcó también otros 4 no relacionados.	Se eliminaron las 4 filas falsas de DATABASECHANGELOG_BOOKING y se re-ejecutó liquibase:update.	Cerrada

N°	Hallazgo	Causa raíz	Acción correctiva ejecutada	Estado
C-10	Toda operación de reservas devolvía 401 'token no incluye personId'.	Copia local de rtm-iam desactualizada; la rama develop ya incluía el claim personId en el JWT.	Se actualizó el entorno local con <code>git merge --ff-only origin/develop</code> (incluyó config SMTP y migración <code>person.image_url</code>).	Cerrada — responsabilidad de infraestructura local, no de código de producto

5.2 Acciones Preventivas

N°	Riesgo que se busca prevenir	Acción preventiva	Dónde se aplica
P-01	Repetir el error de C-09 (changelogSync marcando de más).	Ejecutar <code>liquibase status</code> y revisar <code>changeset</code> por <code>changeset</code> antes de usar <code>changelogSync</code> ; nunca aplicarlo por lote.	Los 4 repos *-db (iam, fleet, booking, payment)
P-02	Repetir el bug de rotación de refresh token (C-04) en nuevos clientes.	Documentar explícitamente, en la guía de integración del cliente, que el refresh token es de un solo uso y debe persistirse tras cada renovación.	Guías de integración para nuevos clientes (web, móvil, futuros)
P-03	Entornos locales desincronizados que oculten bugs reales de backend (como C-10).	Sincronizar cada checkout local con <code>origin/develop</code> antes de iniciar una sesión de pruebas de integración.	Flujo de trabajo de todo el equipo
P-04	Nuevos endpoints de cliente con payloads mal formados (como C-01/C-02).	Validar la forma exacta del payload (nombres de campo, camelCase) contra el contrato real del backend antes de integrarlo, no contra supuestos.	Desarrollo de nuevas features en web y móvil
P-05	Condición de carrera de reservas pase desapercibida en producción.	Agregar prueba de concurrencia (dos reservas simultáneas al mismo vehículo/fechas) a la batería de pruebas antes de cualquier despliegue real.	rtm-booking-reservation

5.3 Acciones de Mejora

N°	Oportunidad de mejora (no es un bug)	Acción de mejora ejecutada / propuesta	Estado
M-01	<code>rtm-docs/15-project-control/risks.md</code> tenía la matriz lista pero sin riesgos reales registrados.	Se completó el registro con 5 riesgos activos reales (condición de carrera, cero CI/CD, SQL Server como punto único de fallo, rechazo de tienda móvil, deriva documentación-vs-código) y 2 lecciones aprendidas de incidentes ya ocurridos (<code>changelogSync</code> , JWT sin <code>personId</code>).	Cerrada

N°	Oportunidad de mejora (no es un bug)	Acción de mejora ejecutada / propuesta	Estado
M-02	Cero pipelines de CI/CD en los 11 repositorios del proyecto.	Pendiente: implementar al menos un workflow básico (build + tests) en GitHub Actions por repositorio. Registrado formalmente como riesgo R-002.	Pendiente
M-03	No existe protección real contra doble reserva del mismo vehículo/fechas (condición de carrera, ver P-05).	Pendiente: agregar <code>@Transactional</code> y una restricción única a nivel de base de datos en <code>booking.reservation</code> . Registrado formalmente como riesgo R-001.	Pendiente
M-04	<code>rtm-fleet-maintenance-db</code> no tenía Dockerfile ni <code>docker-compose.yml</code> , a diferencia del resto de repos <code>*-db</code> .	Se agregaron <code>Dockerfile</code> , <code>docker-compose.yml</code> y <code>.env.example</code> siguiendo el mismo patrón que <code>rtm-booking-db/rtm-payment-billing-db</code> ; se actualizó el README con la sección "Con Docker" y el orden de arranque.	Cerrada
M-05	<code>rtm-docs/10-devops/local-setup.md</code> y <code>environments.md</code> eran plantillas genéricas (Node/Postgres/Redis/K8s/Vault) sin relación con el stack real.	Se reescribieron ambos documentos con el stack real (Java/Spring Boot, SQL Server, Docker Compose), el orden de arranque de los 10 repos, las variables de entorno reales, los incidentes reales como sección de problemas comunes, y el estado honesto de CI/CD y del entorno de producción.	Cerrada

6. ROLES Y RESPONSABILIDADES

- **Equipo de desarrollo:** detecta, registra y ejecuta las acciones correctivas/preventivas/de mejora en su componente.
- **Líder técnico / revisor:** valida la clasificación de prioridad y aprueba el cierre de cada acción tras verificar su eficacia.
- **Todo el equipo:** responsable de alimentar este registro — no es responsabilidad de una sola persona mantenerlo actualizado.